# Reviewer Pack — Algebraic Circuit Complexity via Monodromy Action on Tseitin…

Entry 9330b95a0c44 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 19:56:27 UTC

## Algebraic Circuit Complexity via Monodromy Action on Tseitin Formulas

**Entry ID**: `9330b95a0c44` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 19:56:27 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Topology **Field B** (complexity object): DPLL Search Tree

**Statement**:

> For a Tseitin formula derived from a graph G with n vertices, the DPLL search tree depth is at least 2^{Ω(μ(G))}, where μ(G) is the minimal number of generators of the fundamental group $\pi_1(G)$ under the monodromy action on the universal cover.

**Rationale (proposer's reasoning)**:

> Monodromy actions encode topological complexity via group actions on covering spaces. Linking this to DPLL depth provides a novel geometric measure of proof complexity, as the universal cover's structure governs backtrack resistance in resolution.

**Taxonomy category**: `FOURIER_ANALYTIC` (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `4b9fc33844d94374`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> default: >=80% seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.95 | UNCERTAIN | SAFE |
| NATURAL_PROOFS | SAFE | 0.95 | SAFE | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.00 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.95 | SAFE | SAFE |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda k: abs(A[k][i]))
            A[i], A[max_row] = A[max_row], A[i]
            factor = 1 / A[i][i]
            A[i] = [x * factor for x in A[i]]
            for j in range(m):
                if i != j:
                    factor = A[j][i]
                    A[j] = [A[j][k] - factor * A[i][k] for k in range(n)]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def is_prime(num):
        if num <= 1:
            return False
        for i in range(2, int(math.sqrt(num)) + 1):
            if num % i == 0:
                return False
        return True

    def generate_random_graph(n):
        graph = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    graph[i][j] = graph[j][i] = 1
        return graph

    def fundamental_group_generators(G):
```

```
    n = len(G)
    edges = [(i, j) for i in range(n) for j in range(i + 1, n) if G[i][j]]
    generators = []
    visited = [False] * n
    stack = [0]
    while stack:
        node = stack.pop()
        if not visited[node]:
            visited[node] = True
            for neighbor in range(n):
                if G[node][neighbor] and not visited[neighbor]:
                    stack.append(neighbor)
                    generators.append((node, neighbor))
    return len(generators)

def dpll(Tseitin_formula):
    def solve(formula, assignment):
        if not formula:
            return True
        literal = next(l for l in formula if isinstance(l, int) or l[0] != '¬')
        pos_var = abs(literal)
        neg_var = -pos_var
        if pos_var in assignment and assignment[pos_var]:
            return solve(formula, assignment)
        elif neg_var in assignment and not assignment[neg_var]:
            return solve(formula, assignment)
        else:
            assignment[pos_var] = True
            if solve(formula, assignment):
                return True
            assignment[pos_var] = False
            assignment[neg_var] = True
            if solve(formula, assignment):
                return True
            assignment[neg_var] = False
            return False

    n = len(Tseitin_formula)
    assignment = {}
    return solve(Tseitin_formula, assignment)

def tseitin_formula(G):
    n = len(G)
    Tseitin_formula = []
    for i in range(n):
        Tseitin_formula.append((i + 1,))
    for i in range(n):
        for j in range(i + 1, n):
            if G[i][j]:
                Tseitin_formula.append(('¬', i + 1, '¬', j + 1))
                Tseitin_formula.append((i + 1, j + 1))
    return Tseitin_formula

def simulate_dpll(G):
    Tseitin = tseitin_formula(G)
    depth = 0
```

```
        stack = [Tseitin]
        while stack:
            formula = stack.pop()
            if not formula:
                break
            literal = next(l for l in formula if isinstance(l, int) or l[0] != '¬')
            pos_var = abs(literal)
            neg_var = -pos_var
            if pos_var in assignment and assignment[pos_var]:
                continue
            elif neg_var in assignment and not assignment[neg_var]:
                continue
            else:
                assignment[pos_var] = True
                stack.append(formula)
                depth += 1
        return depth

    n = random.randint(5, 40)
    G = generate_random_graph(n)
    μ_G = fundamental_group_generators(G)
    Tseitin = tseitin_formula(G)
    depth = simulate_dpll(G)

    conjecture_holds = depth >= 2 ** (0.1 * μ_G)
    counterexample = "" if conjecture_holds else f"Graph with {n} vertices and μ(G)={μ_G}, DPLL de

    return {
        "metric_name": "DPLL Depth",
        "metric_value": depth,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_depth = sum(r["metric_value"] for r in results) / len(results)
    std_depth = math.sqrt(sum((r["metric_value"] - mean_depth) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_depth} std={std_depth} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ef54ee1.py", line 157, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ef54ee1.py", line 138, in run_trial
    depth = simulate_dpll(G)
            ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ef54ee1.py", line 122, in simulate_dpl
    pos_var = abs(literal)
              ^^^^^^^^^^^^
TypeError: bad operand type for abs(): 'tuple'
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> Test crashed due to TypeError, preventing depth measurement. Support condition cannot be evaluated without successful trials. | next: Fix the simulate_dpll function to handle tuple literals properly and re-run experiments

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | qwen3:8b | 0 | 0 | 90141 |
| 2 | propose | ollama_remote | qwen3:8b | 0 | 0 | 101963 |
| 3 | propose | ollama_remote | qwen3:8b | 0 | 0 | 81919 |
| 4 | preregistration | ollama_remote | qwen3:8b | 0 | 0 | 24114 |
| 5 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 20807 |
| 6 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 15385 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13179 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 16008 |
| 9 | judge | ollama_remote | qwen3:8b | 0 | 0 | 16578 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 380095 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/9330b95a0c44.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9330b95a0c44.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/9330b95a0c44.tar.gz` (if generated)